Tutorial: Reverse Engineering, Part 2

Ibrahim Konare, Ryan Evans, Boyang Wang

Data and System Security Lab (DaSec), Department of ECE, University of Cincinnati
Last Modified: December, 2025, Status: Completed

1 Introduction

What This Document is About. This document covers creating a new project in Ghidra, navigation and comprehension of Ghidra's multi-panel user interface, going through a binary analysis overview with a simple introductory binary example compiled from a C++ program, and establishing the fundamentals of reverse engineering techniques (e.g., how to find a hard-coded password and how to patch binary to change the behavior of a program) in general. In addition, we also describe how to perform these steps with Binary Ninja, another popular reverse engineering tool.

Pre-Requisites. Our tutorial document, named Revere Engineering Part 1, describes how to install a Virtual Machine (VM) and Ghidra. We suggest that readers review it first. Before proceeding with our Ghidra analysis, we need to set up our example file. Please download the following code from our GitHub repository onto the installed VM with the following CLI command:

```
$ curl -O https://raw.githubusercontent.com/UCdasec/Tutorial-Ghidra/main/src/password_example1.cpp
```

GitHub Repository. Our tutorial document (Part 1, Part 2, Part 3) as well as the code and tutorial videos can be found on our GitHub repository[1]. Both Part 2 and Part 3 focus on similar reverse engineering process, where Part 2 focuses on a C++ program while Part 3 is based on a C program.

[1] https://github.com/UCdasec/Tutorial-Ghidra

2 Password Example in C++

To demonstrate the effective use of Ghidra, it is very important to understand the compilation process, i.e., the process of transforming high-level code (e.g., C code) into machine code. In our first example, we have a program, named `password_example1.cpp`, written in C/C++. This program contains 33 lines in total. Given this program, a user is asked to enter a password until it matches the correct one ("`B3@rCatz`") pre-defined in the code. When a user enters the correct password, the program calls a hidden function `secret_function` that displays "You have completed your first RE!" and will be prompt to exit. Otherwise, if the input is incorrect, the loop will not break and display "Wrong password. Try Again" The code snippet is presented below.

```cpp
// password_example1.cpp
#include <iostream>
#include <string>

void secret_function() {
    std::cout << " You have completed your first RE!\n";
}

bool validate_password(const std::string &input) {
    std::string pass = "B3@rCatz";  // pre-defined password
    return input == pass;
}

int main() {
    std::string user_input;

    while (true) {
        std::cout << "\nEnter the magic word: ";
        std::cin >> user_input;

        if (validate_password(user_input)) {
            std::cout << "Correct!";
            secret_function();

            std::cin.ignore();
            std::cout << "Press Enter to exit...";
            std::cin.get();
            break;
        }
        else {
            std::cout << "Wrong password. Try Again :( \n\n";
        }
    }
    return 0;
}
```

Using a compiler, such as GNU Compiler Collection (GCC), we can convert human readable code to machine instructions that our target processor can understand. Usually, most IDEs will come equipped with a compiler installed and one can generate a binary with machine instructions by simply clicking a button in an IDE to complete the process, where the IDE runs the commands behind the scene. To make the process more explainable, we compile our code directly and generate an executable file (`Ghidra_EX_1`) by running the command below in the terminal:

```
$ g++ -static -o Ghidra_EX_1 password_example1.cpp
```

This executable file is a binary file that contains all the machine instructions generated by the C/C++ program. The `-static` parameter is used to tell the compiler to insert all the libraries in the binary that will be generated. The `-o` parameter lets us name our file to `Ghidra_EX_1` (-o for Output). And lastly, at the end is the name of the source code file being compiled.

It is important to understand that compiling any program is a multi-stage process with the following steps:

- **Preprocessing** – regarding C/C++, any directives that begin with # (ex. #include, #define, #ifdef, #ifndef) is processed, inserting the contents from the header files, replacing and expanding macros and constants, and provide conditional compilation respectively.
- **Compilation** – During this step the preprocessed code is translated into assembly using various theories, breaking code into tokens (Lexical analysis), check grammar and structure of the code (Syntax analysis), ensure logic and types are correct (Semantic Analysis), and lastly provides optimization to the code to improve performance. Note: any errors in code would typically be detected in this stage.
- **Assembly** – Turns assembly code into object code (machine code, binary opcodes), e.g., an object file (.o).
- **Linking** – combines multiple object files and external libraries into a complete program. This stage will output a single executable file.

When we only have a binary file, either an object file or executable file, but no access to the source code, software reverse engineering can perform the analysis on the binary and help us understand the functionalities and behavior of this binary before we use it or integrate it into a system. Ghidra offers a powerful decompiler and a range of various analyzers to predict what the source code and function may look like.

2.1 Initial Binary Analysis

Assume one has compiled the source code `password_example1.cpp` and generate the binary `Ghidra_EX_1` successfully or downloaded the binary directly from our GitHub repo, we can run the binary first with the command below to observe any distinctive behavior, identify any issues, and determine what actions we can take.

```
Ghidra_EX_1: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically linked, BuildID[sha1]=03643188ad662b
c8a64c09caaeb90dc5c3bb1e32, for GNU/Linux 3.2.0, not stripped
```

Fig. 1: After running command **file Ghidra_EX_1**

```
$ ./Ghidra_EX_1
```

After running it, we can see that it is asking for a specific input string as a magic word. Once we type a random string, the program asks for a new input again, which indicates the magic word is a pre-defined string (e.g. password) by the program and the one we just provide does not match to the password.

Once we have the initial attempt above with the program, we can first utilize a couple of command-line tools to obtain some basic information for reverse engineering on this program. We can first run `file <filename>`, in our case `file` is `Ghidra_EX_1`, then we will see a description of that file.

```
$ file Ghidra_EX_1
```

After running the file command above, we can see that the file is ELF (Executable and Linkable Format) formatted, 64-bit, x86 architecture, sha1 hash for integrity, and lastly it is not stripped (i.e., a binary with headers and debug information). This information is helpful before using Ghidra.

Next, we can run command `strings <filename>`, which will extract sequences of printable characters from this binary. It is helpful to detect function and variable names in the binary. One should also keep in mind that this command will also print function names from libraries, which may not be specifically useful to the target program.

```
$ strings ./Ghidra_EX_1 | head -n 15
```

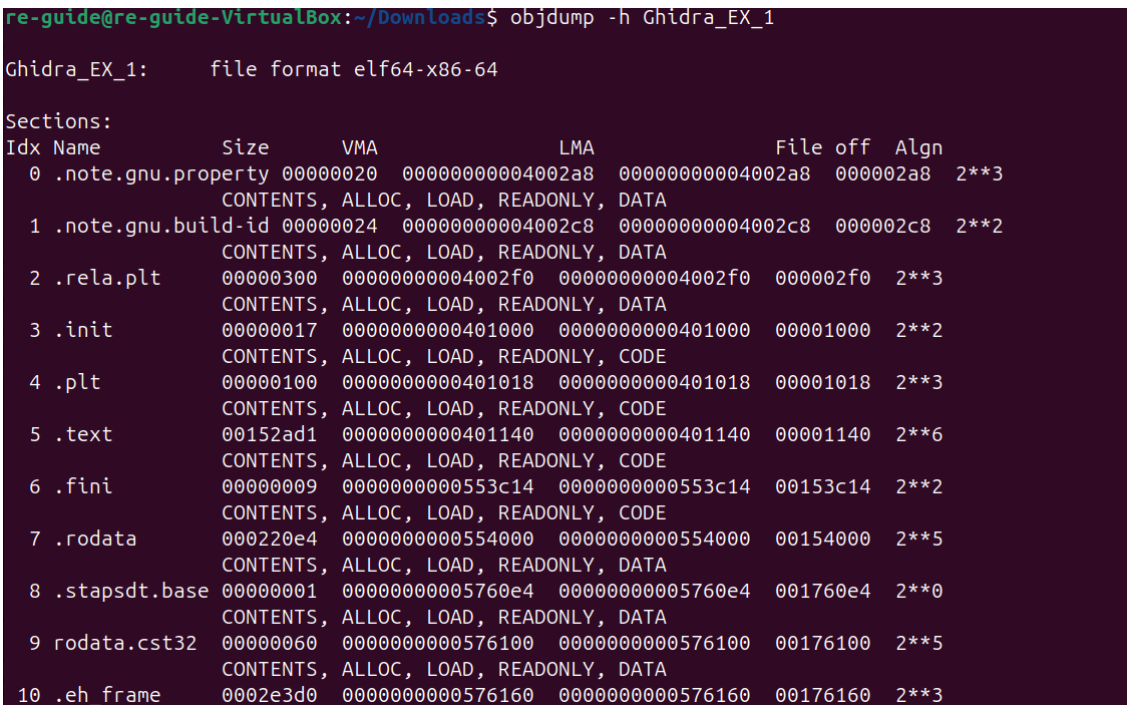
```
❯ strings ./Ghidra_EX_1 | head -n 15
<$I9
|$0H
D$@H9
D$0H
D$@H
D$ H
D$ H
<$H9
<$H9
<$H9
AVAUL
D$(1
PTE1
u+UH
ATSH
```

Fig. 2: After running command **strings Ghidra_EX_1**

In addition, we can also run `objdump` or `hexdump`, particularly with parameters like `-h`. This command allows us to inspect section headers, raw file type, the structure of a binary. *It is worth mentioning that all these commands are static, i.e., we obtain the information of a binary without executing it.*

```
$ objdump -h  Ghidra_EX_1
```

Another useful command is `strace/ltrace`. Either command will give an outline of what system processes are called associated with a binary. Specifically, `strace` runs the program and intercepts and records all system calls made by the running program until the program is stopped. This is a dynamic tool, meaning the binary will be executed while we run this command.



```
re-guide@re-guide-VirtualBox:~/Downloads$ objdump -h Ghidra_EX_1

Ghidra_EX_1:     file format elf64-x86-64

Sections:
Idx Name          Size      VMA               LMA               File off  Algn
  0 .note.gnu.property 00000020  00000000004002a8  00000000004002a8  000002a8  2**3
                    CONTENTS, ALLOC, LOAD, READONLY, DATA
  1 .note.gnu.build-id 00000024  00000000004002c8  00000000004002c8  000002c8  2**2
                    CONTENTS, ALLOC, LOAD, READONLY, DATA
  2 .rela.plt     00000300  00000000004002f0  00000000004002f0  000002f0  2**3
                    CONTENTS, ALLOC, LOAD, READONLY, DATA
  3 .init         00000017  0000000000401000  0000000000401000  00001000  2**2
                    CONTENTS, ALLOC, LOAD, READONLY, CODE
  4 .plt          00000100  0000000000401018  0000000000401018  00001018  2**3
                    CONTENTS, ALLOC, LOAD, READONLY, CODE
  5 .text         00152ad1  0000000000401140  0000000000401140  00001140  2**6
                    CONTENTS, ALLOC, LOAD, READONLY, CODE
  6 .fini         00000009  0000000000553c14  0000000000553c14  00153c14  2**2
                    CONTENTS, ALLOC, LOAD, READONLY, CODE
  7 .rodata       000220e4  0000000000554000  0000000000554000  00154000  2**5
                    CONTENTS, ALLOC, LOAD, READONLY, DATA
  8 .stapsdt.base 00000001  00000000005760e4  00000000005760e4  001760e4  2**0
                    CONTENTS, ALLOC, LOAD, READONLY, DATA
  9 rodata.cst32  00000060  0000000000576100  0000000000576100  00176100  2**5
                    CONTENTS, ALLOC, LOAD, READONLY, DATA
 10 .eh_frame     0002e3d0  0000000000576160  0000000000576160  00176160  2**3
```

Fig. 3: After running command **objdump -h Ghidra_EX_1**

Once we have obtained the basic information of a binary with the commands above, we are ready to move on to the next phase of reverse engineering. Specifically, we will utilize Ghidra to extract hard-coded secret code and manipulate the binary in a way in which any input from the user would be accepted.

```
re-guide@re-guide-VirtualBox:~/Downloads$ strace ./Ghidra_EX_1
execve("./Ghidra_EX_1", ["./Ghidra_EX_1"], 0x7fff56074ba0 /* 48 vars */) = 0
brk(NULL)                               = 0x40ae000
brk(0x40aed80)                          = 0x40aed80
arch_prctl(ARCH_SET_FS, 0x40ae400)      = 0
set_tid_address(0x40ae6d0)              = 19075
set_robust_list(0x40ae6e0, 24)          = 0
rseq(0x40aed20, 0x20, 0, 0x53053053)    = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
readlinkat(AT_FDCWD, "/proc/self/exe", "/home/re-guide/Downloads/Ghidra_"..., 4096) = 36
getrandom("\x8d\xef\xfa\xe2\x7f\x24\xd3\xc6", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0x40aed80
brk(0x40cfd80)                          = 0x40cfd80
brk(0x40d0000)                          = 0x40d0000
mprotect(0x5aa000, 45056, PROT_READ)    = 0
futex(0x5b7e1c, FUTEX_WAKE_PRIVATE, 2147483647) = 0
fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0), ...}) = 0
write(1, "\n", 1
)                        = 1
write(1, "Enter the magic word: ", 22Enter the magic word: )  = 22
fstat(0, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0), ...}) = 0
read(0, testVal
"testVal\n", 1024)              = 8
write(1, "Wrong password. Try Again :( \n\n", 31Wrong password. Try Again :(

) = 31
write(1, "\n", 1
)                        = 1
write(1, "Enter the magic word: ", 22Enter the magic word: )  = 22
read(0,
```

Fig. 4: After running command **strace ./Ghidra_EX_1**

3 Creating a Ghidra Project

In this section, we will go over the first steps into using Ghidra. We will first create a project on Ghidra for the analysis of the target binary. To make a new project, go to **File** → **New Project** and select **non-shared project** (since we will be working independently, not collaboratively, in this example).

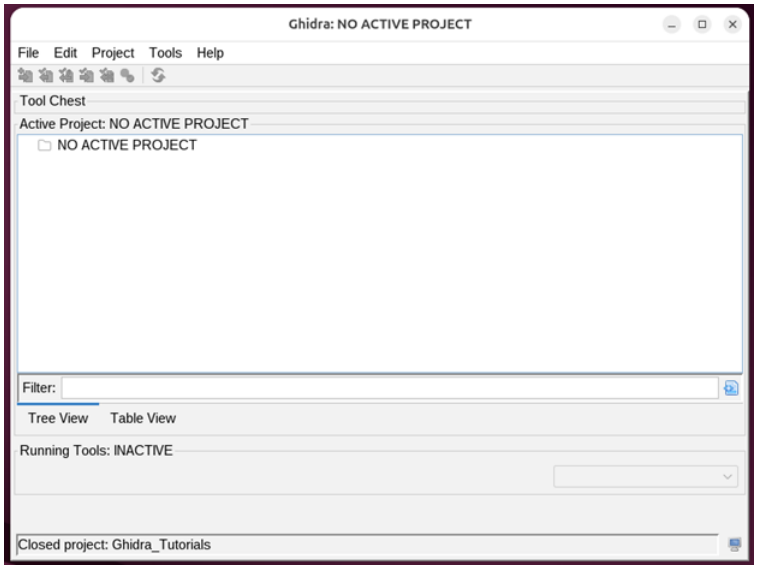


Fig. 5: New Project Window

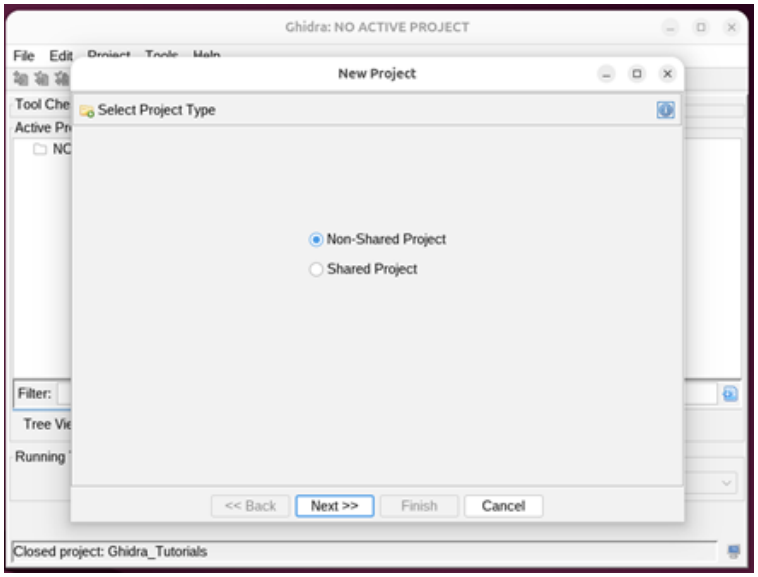


Fig. 6: New Project Window

Next, click on **File** → **Import File** and browse to the target binary (`Ghidra_EX_1`). We will leave the following settings as they are since the data is accurate (as we observed by running the `file` command). Once it is completed, we can observe the summary window regarding this binary target.

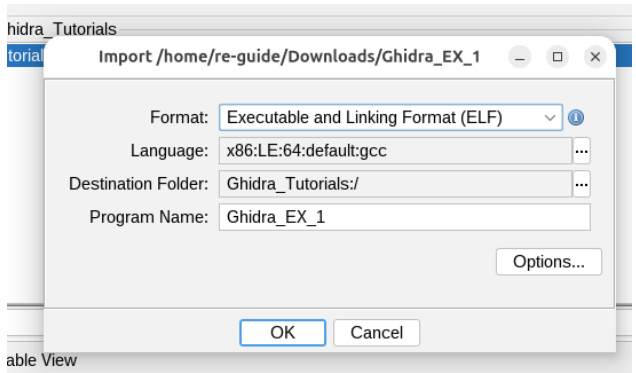


Fig. 7: New Project Window

Fig. 8: Summary Window after Adding the Binary

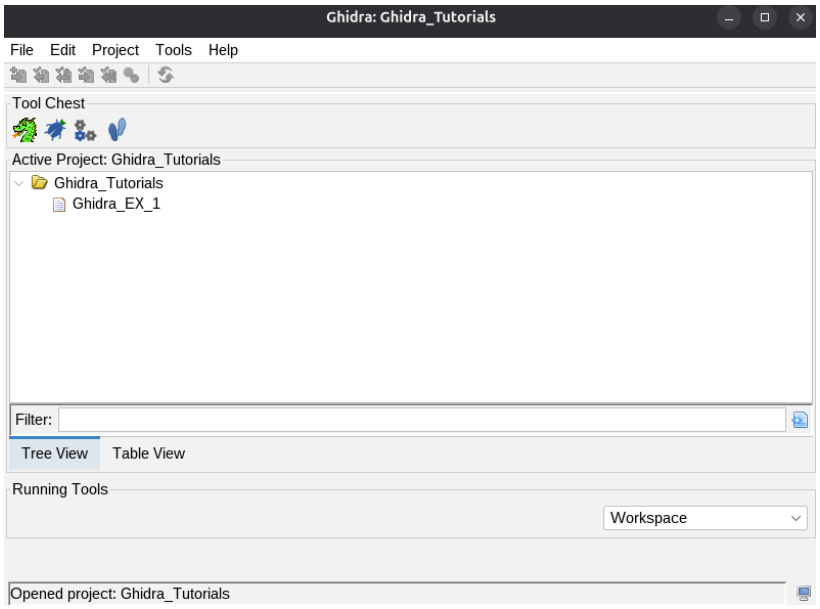


Fig. 9: Project View after Adding the Binary

4 Exploring Ghidra Windows

Once we add the binary, we can start analyzing it with Ghidra. Ghidra provides a list of different analyzers that one can use.

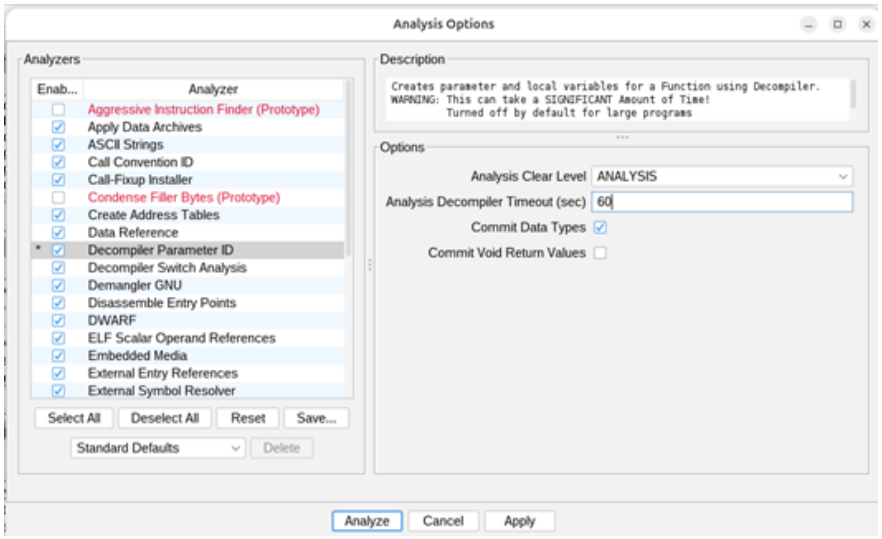


Fig. 10: Analysis Window

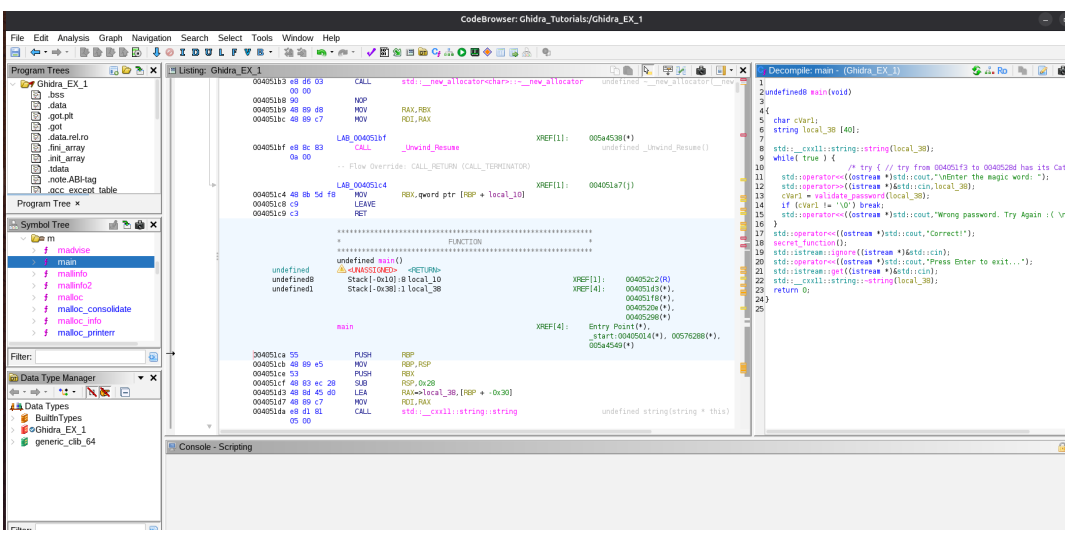


Fig. 11: Ghidra Main GUI

Once the previous setup steps are done, Ghidra will automatically open the Code browser. In the Code browser we can see several windows pertaining to our binary. Here is a breakdown of the windows that are numbered:

1. **Program Trees**: in this panel, we can see a breakdown of our ELF file binary segments. It is helpful to identify data regions and understand how the binary is loaded into memory.
2. **Symbol Tree**: Structure overview of the components in the binaries such as functions, classes, labels, variables, etc. It serves as a fast way to navigate to each area of the binary.
3. **Data Type Manager**: All data types that were captured during analysis will be organized and stored here. This includes primitive (int, double, float) types, structures, pointers, enums, etc. In this section, we will be able to manipulate data types if needed.
4. **Listing Window**: The core of our low-level analysis. It displays the disassembled machine instructions of the binary. From the leftmost side it lists the following:

```
                    LAB_004051df                                    XREF[1]:     0040528e(j)
004051df 48 8d 05        LEA        RAX,[s_Enter_the_magic_word:_0055405d]          = "\nEnter the magic word: "
         77 ee 14 00
004051e6 48 89 c6        MOV        RSI=>s_Enter_the_magic_word:_0055405d,RAX       = "\nEnter the magic word: "
004051e9 48 8d 05        LEA        RAX,[std::cout]                                 = ??
```

Fig. 12: Snippet of Machine Code

The above line contains the following fields of information

- Address field → memory address where data is located (`0x004051df`)
- Bytes → opcode representing the instructions (`48 8d 05`)
- Mnemonic → instruction disassembled from the opcode (`LEA`)
- Instruction → machine instructions sometimes will show the output like in the picture above (`RAX,[s_Enter the_magic_word:_0055405d]`)
- XREF → usually place at the beginning of each section header, generated by Ghidra to show other addresses referencing the current location or instructions.

```
        XREF[1]:     004052c2(R)
        XREF[4]:     004051d3(*),
                     004051f8(*),
                     0040520e(*),
                     00405298(*)
XREF[4]:     Entry Point(*),
             _start:00405014(*), 00576288(*),
             005a4549(*)
```

Fig. 13: XREF Header

5. **Decompile Window**: In this window we can see the generated C code Ghidra has created. It is worth mentioning that this generated C code typically does not exactly match the original source code of the binary during the reverse engineering process.
6. **Tool Bar**: Panel consists of various tools to aid analysis process, such as redo and undo.
7. **Scripting Window**: allows automation of task and customize analysis.

There are many other powerful panels listed under the **Windows** tab and we will skip the remaining ones fow now.

5 Revealing Hard-Coded Password

5.1 Analyzing the Main Function

Once we are familiar with the user interface, it is time to go over our first Ghidra crack. The first thing to always start with is to look for the main function of the binary using the symbol tree. This is usually a great starting place for reverse engineering.

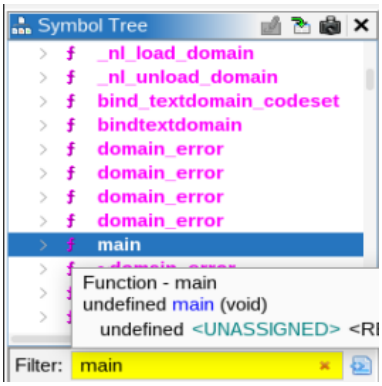


Fig. 14: Main function hovered in Symbol Tree Window

After double clicking on the main function, the Decompile window will show the main function code in C. **Note:** Since the original program was coded in C++, at first glance the C code may not look understandable. However, with some basic efforts, anyone familiar with either language should be able to read the code.

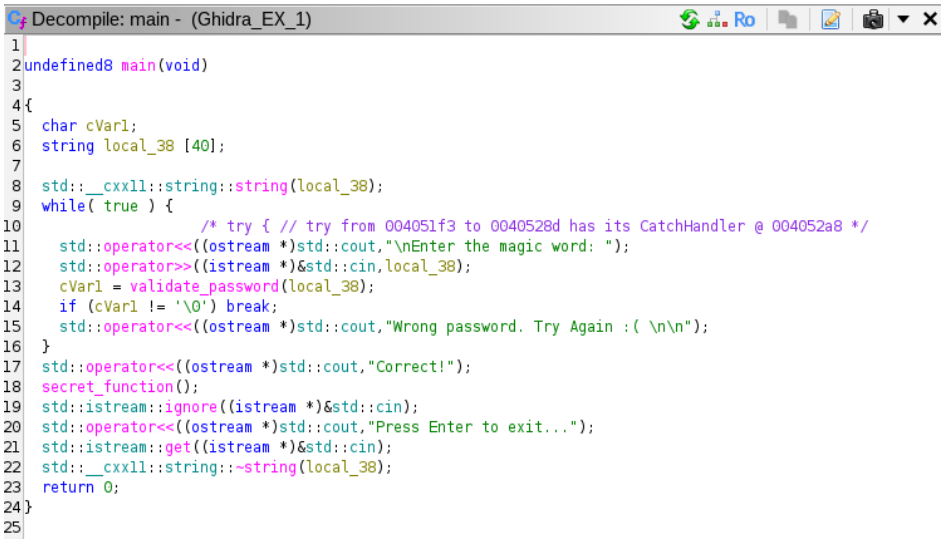


Fig. 15: Decompiled Code of function Main

In the de-compiled code, we can observe several key components. First, there is a while loop that continuously executes the programs logic until a satisfactory point is reached, in our case if `cVar1 != '\0'`. Next, the program has several user interactions, outputting "Enter the magic word" to screen, and prompting them to input. This input is stored in the variable called `local_38` and it is called in function `validate_password`. Lastly if `validate_password` returns a 0 (since `cVar1` is equal to the function), the program will exit after running `secret_function`.

5.2 Examining the Secret Function

Starting from the last function called `secret_function`, we can examine its content by double clicking, which results in the code shown in the image below.

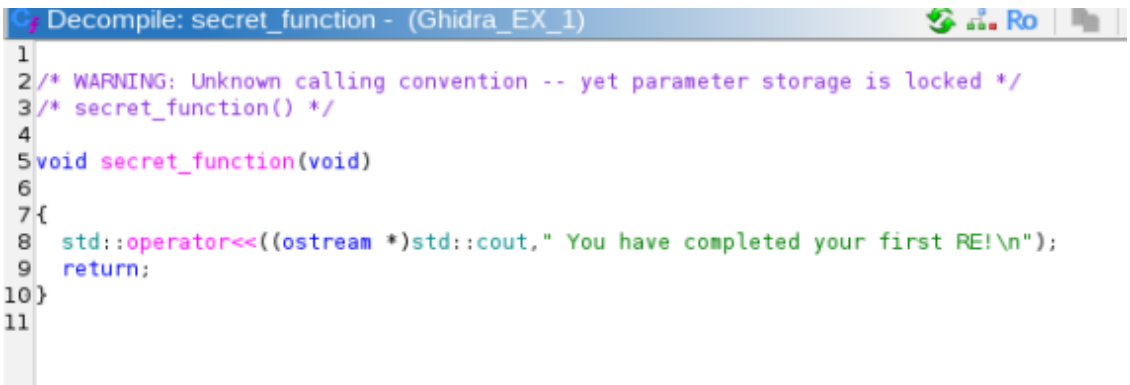


Fig. 16: Decompiled code of function secret_function

From this decompiled code, we can see that the function is quite simple, only containing a single output that prints to the console "You have completed your first RE! \\n". Now we know what will be displayed if we get the password correctly.

5.3 Analyzing the Password Validation

Next, let us check `validate_password` function since the input is called in it. Using the symbol tree, we can go back to main and double click on `validate_password` to see the function. As we look through the `validate_password` function, we can spot a peculiar string "`B3arCatz`", which likely is going to be the hard-coded password.

5.4 Function Signature Improvement

Through our initial observations, it seems that the `validate_password` function should have the return type Boolean rather than `ulong`. As a result, we can go ahead and modify the return type and hope this will make the de-compiled code more clear and readable. Specifically, we will

Decompile: validate_password - (Ghidra_EX_1)

```
1
2 /* validate_password(std::__cxx11::string const&) */
3
4 ulong validate_password(string *param_1)
5
6 {
7   bool bVar1;
8   undefined7 extraout_var;
9   string local_48 [39];
10  allocator local_21;
11  allocator *local_20;
12
13  local_20 = &local_21;
14                    /* try { // try from 00405171 to 00405175 has its CatchHandler @ 004051a9 */
15  std::__cxx11::string::string<>(local_48,"B3@rCatz",&local_21);
16  std::__new_allocator<char>::~__new_allocator((__new_allocator<char> *)&local_21);
17  bVar1 = std::operator==(param_1,local_48);
18  std::__cxx11::string::~string(local_48);
19  return CONCAT71(extraout_var,bVar1) & 0xffffffff;
20 }
21
```

Fig. 17: Snippet of Machine Code

be editing the function signature by right clicking on the function name and replacing `ulong` with `bool`. Also in the meantime, we can change the parameter variable name as we know it is the user input.

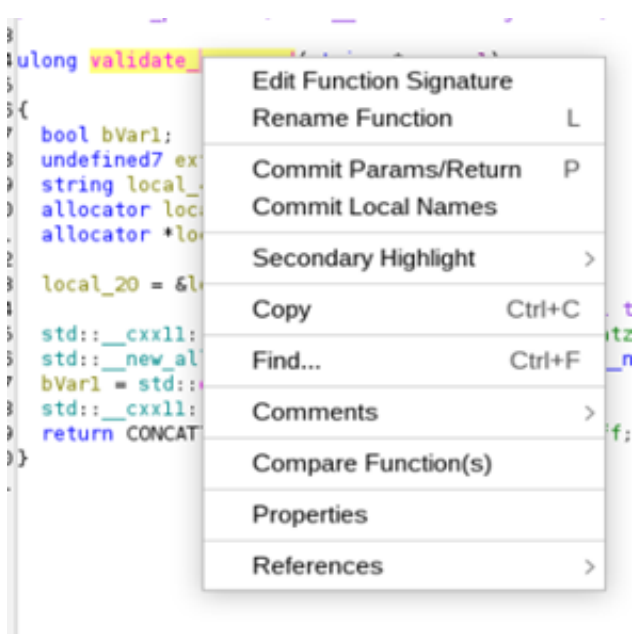


Fig. 18: Edit Function Signature

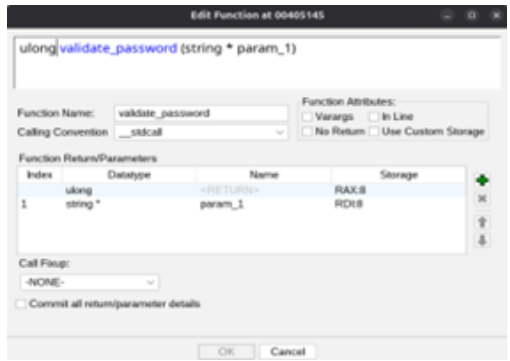


Fig. 19: Edit Function Signature Interface (before change)

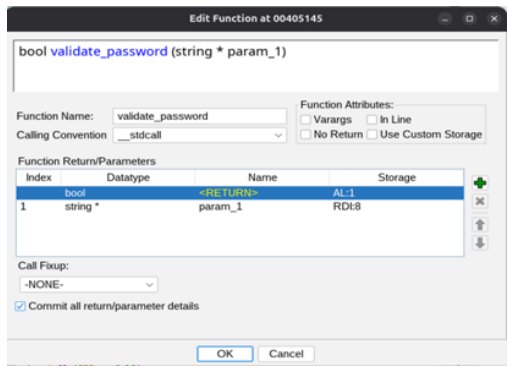


Fig. 20: Edit Function Signature Interface (after change)

Fig. 21: Result in Cleaner Code (after changing the return type of the function)

After changing these variables, our code looks much cleaner. We can see that `userInput` and `local_48` are being compared with each other and stored as a Boolean, `bVar1`. In this case in line

14 we can see that string "`B3@rCatz`" is stored at `local_48`, which confirms that it is the hard-coded magic word. Given the analysis above, we run the program again and provide an input as "`B3@rCatz`" and see what happens.

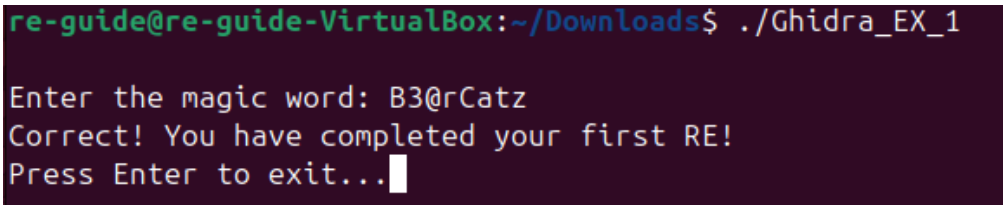


Fig. 22: Binary Cracked with the correct password recovered using Ghidra

6 Patching the Code in Ghidra

In this section, we will try to modify the original binary and bypass the conditional jump instruction in the binary so that any input would be valid (i.e., printing "Correct! You have completed your first RE!"). Going into the main function, we can highlight the related machine code for the `if` statement, which makes the decision of whether a given input being correct. **In essence, we will generate a mutation of the original binary, where the functionality of the mutated binary will be different compared to the original one**.

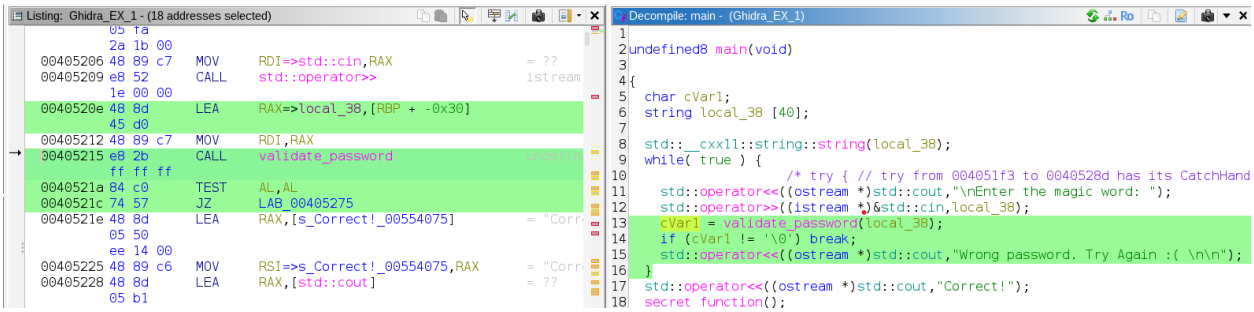


Fig. 23: Highlighted Code related to the Conditional Jump

In the figure above, we can examine that the function `validate_password` is called at address `0x00405215`, and then the program compares whether the register `AL` is zero by using a `test` instruction at address `0x0040521a`. Then, at the next address `0x0040521c`, the program operates a `jz` instruction to jump to `LAB_00405275`. If it does not jump, then at the next address `0x0040521e`, the program prepares a string starting with "Correct!" and stores it at register `RAX`. This indicates, if not jump, then the given password is correct.

Now the question for us is, **what if we can do something to completely skip the jump even if the given password is not correct?** Then, as a result, the program can still print "Correct! You have completed your first RE!"

We can achieve this by replacing the original jump instruction `jz` at address `0x0040521c` with NOP instructions `nop`. The NOP instruction is a no-operation instruction in which the program tells the processor to do nothing and proceed to the next instruction in the stack.

Specifically, we can right click on the line at address `0x0040521c` with bytes `74 57` (or instruction `jz`), and navigate to patch instructions. We replace the two bytes `74 57` with two bytes `66 90`, which stands for a NOP instruction.

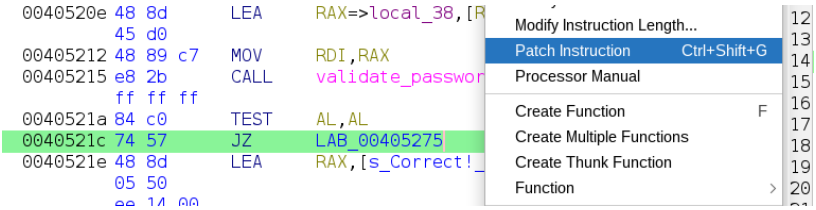


Fig. 24: Patch Instructions (before changes)

Fig. 25: Instructions at address `0x0040521c` (after the changes)

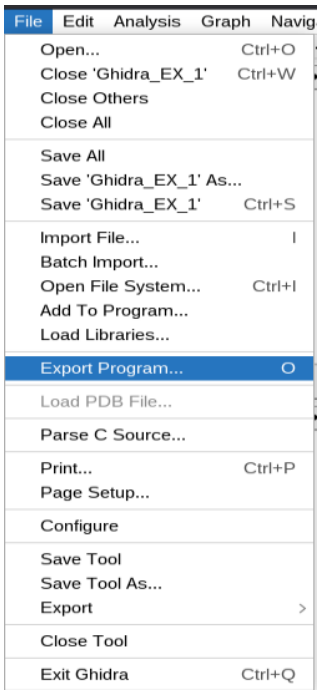


Fig. 26: Export Program

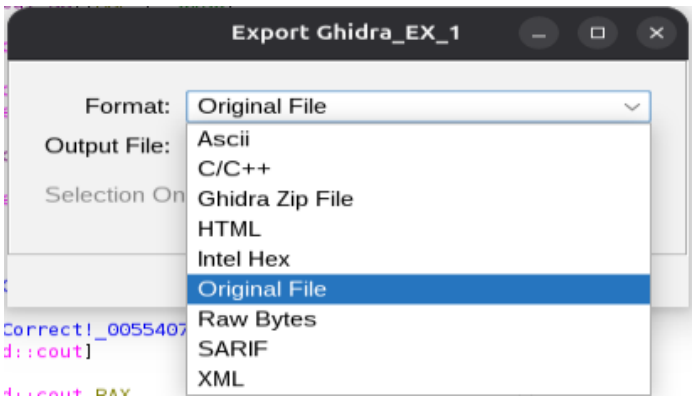


Fig. 27: Snippet of Machine Code

Once the instruction line has been updated, we need to save the binary and test it out.

Click on **File** → **Export Program**, change file type to **original file**, and name as `Ghidra_EX_1_Patch` (or any name you would like choose).

The last step is to verify that our patched binary will work as expected (i.e., passing given any password input). We provide a couple of examples below

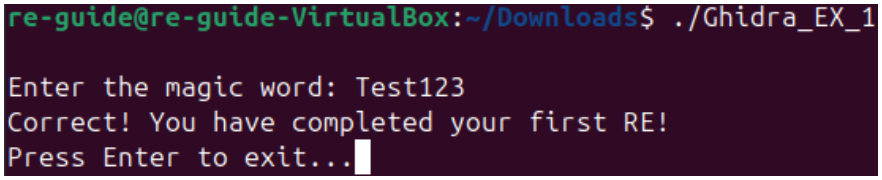


Fig. 28: Run Ghidra_EX_1_Patch with password "Test123"

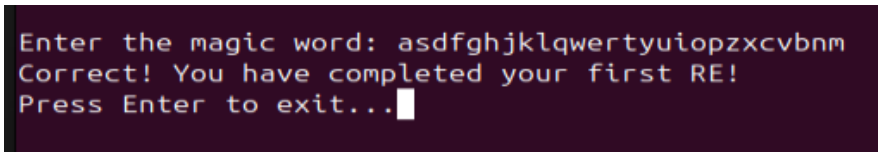


Fig. 29: Run Ghidra_EX_1_Patch with password "asdfghjklqwertyuiopzxcvbnm"

7 Reverse Engineering with Binary Ninja

In this section, we extend our tutorial by repeating the previous steps on the same binary (`Ghidra_EX_1`) to identify the hard-coded password and patch the binary, but with another popular reverse engineering tool, named Binary Ninja.

7.1 Installing Binary Ninja

We use Binary Ninja 5.2, Commercial version (with an education discount)[2]. An installer was provided by the vendor after purchase. We install and setup it by following the installer. Binary Ninja also offers a free version.

7.2 Creating a Project in Binary Ninja

In this section, we will briefly discuss the steps to create our first project in Binary Ninja. When we first run Binary Ninja, our screen will look like in Fig. 30. To create a project, select "New Project" on the left side of the screen, and name our project as "MyFirstBinaryNinja", this will pop up the window seen in Fig. 31. We add the target binary (`Ghidra_EX_1`) and select Okay. Next, we can double click the project and we see the project in Fig. 32.

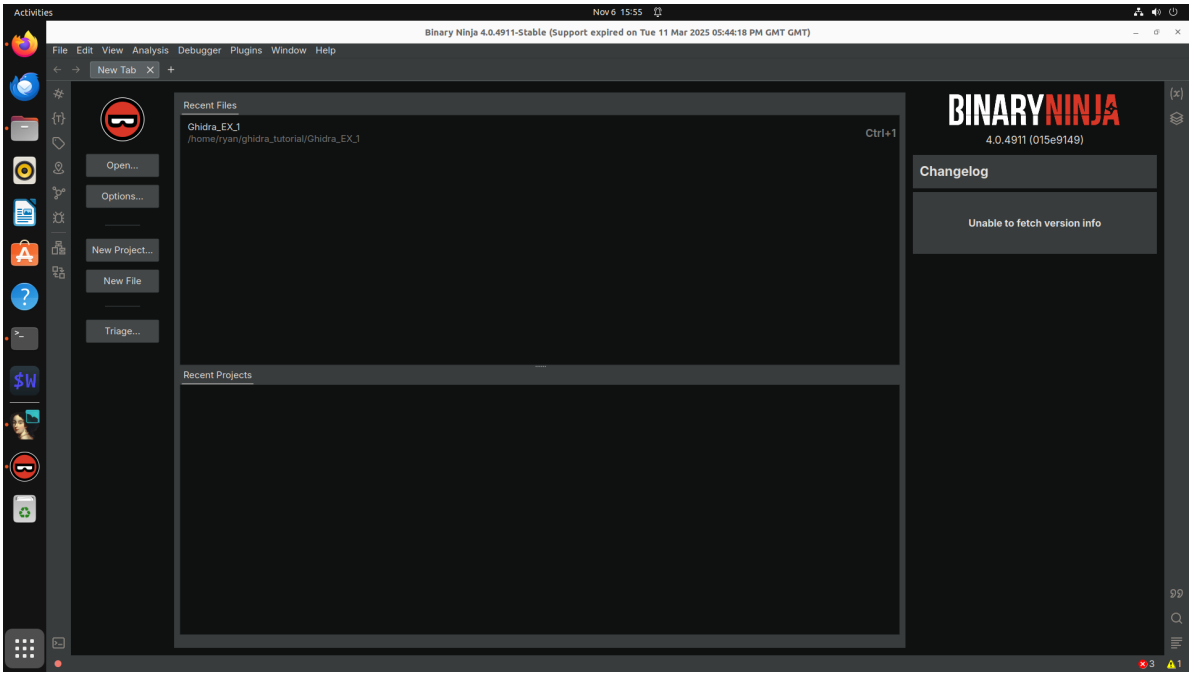


Fig. 30: Create a New Project

[2] https://binary.ninja/purchase/#non-commercial

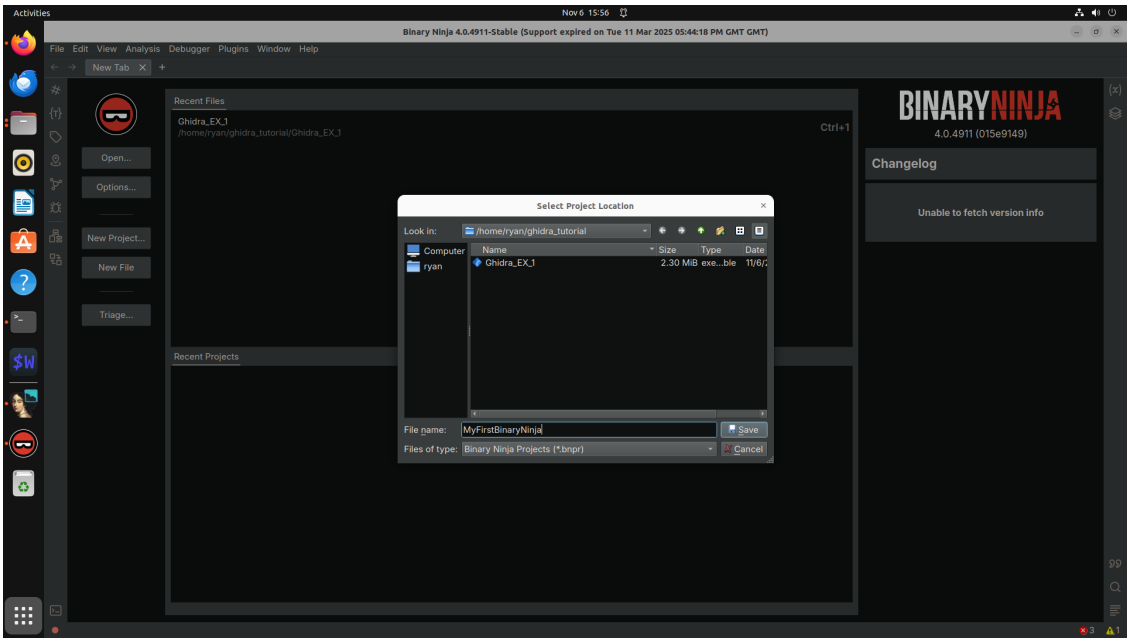


Fig. 31: Add the Target Binary (`Ghidra_EX_1`)

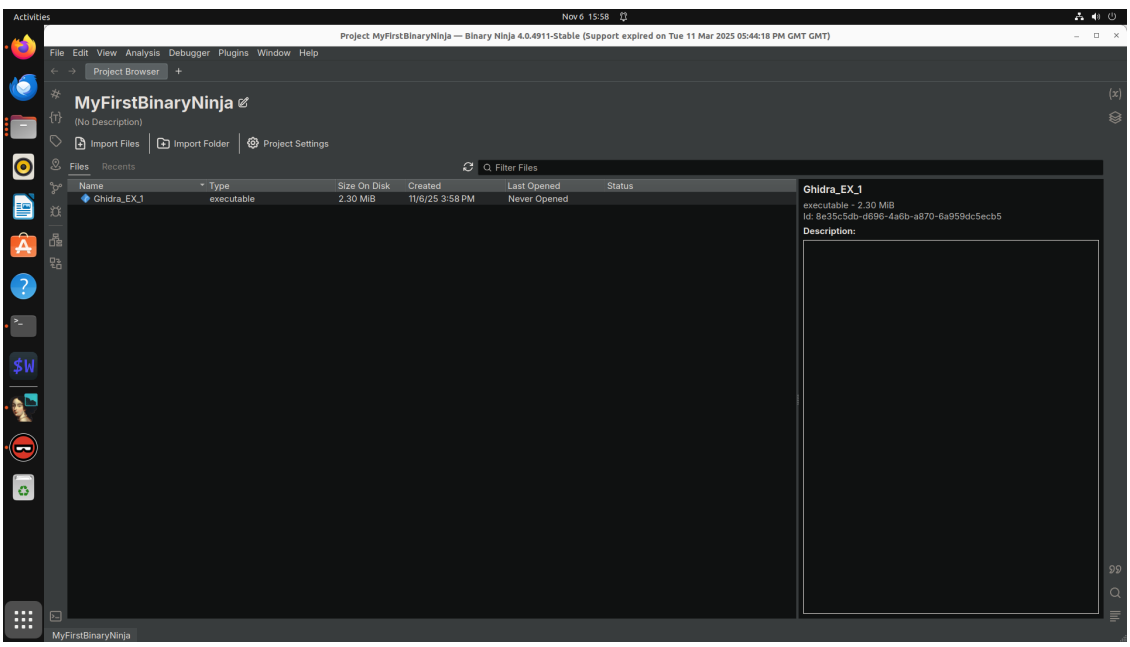


Fig. 32: Open the New Project

7.3 Exploring User Interface Windows in Binary Ninja

Once we add the binary, we can observe the user interface, which is shown in Fig. 33. There are 4 main components to the user interface in Binary Ninja:

1. **Symbol List**: Contains a list of the known program symbols
2. **Cross Reference (XREF)**: Contains reference for functions and memory. For example, a given function will have references to all of its callers and callees.
3. **Main View**: Shows the disassembly of the program
4. **Feature Map**: A heatmap of how Binary Ninja interprets the binary and how confident Binary Ninja is in the analysis.

Beyond these windows, Binary Ninja also offers the tool bar at the top of the program with various utilities to help us conduct binary analysis.

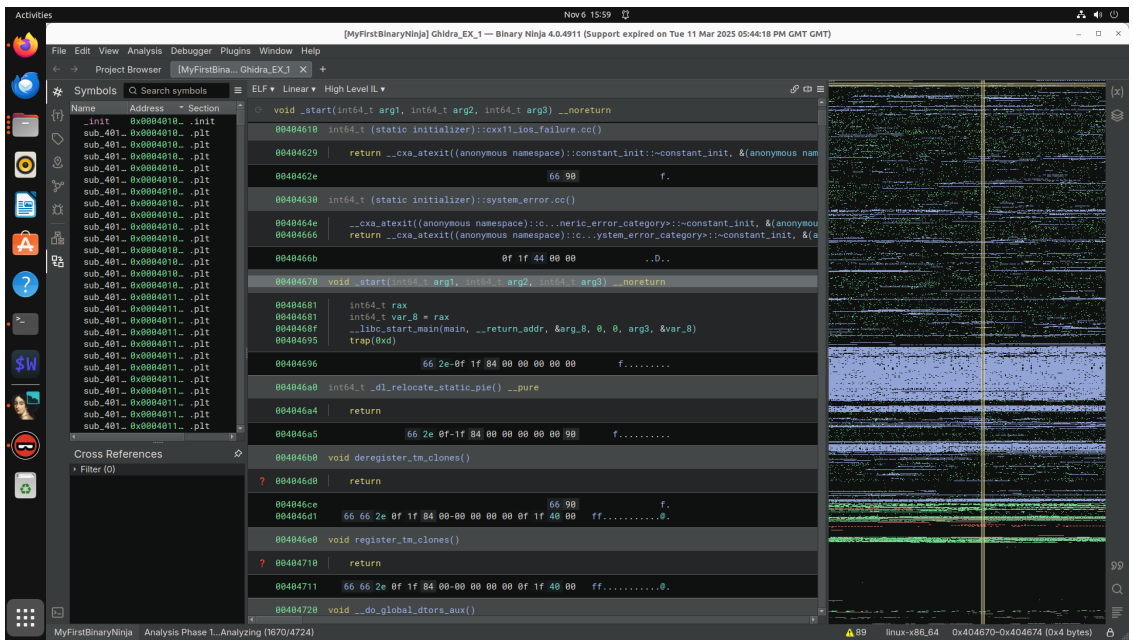


Fig. 33: User Interface in a Project

7.4 Analyzing the Main Function

Now we will consider the disassembly of the main function. In Fig. 34 we see what Binary Ninja thinks the source code of the main function looks like. Upon initial inspection, we can see that this is relatively similar to the actual main function. Again, we can see that the user is prompted for a password, which is then checked by the function validate_password, and if the password is incorrect they stay in the while loop, but if the password is correct we break out of the loop, run the function named secret_function and eventually exit.

```
00404869  int64_t main()

00404876      void* fsbase
00404876      int64_t rax = *(fsbase + 0x28)
0040488c      void var_48
0040488c      std::string::string(&var_48)
004048a5      while (true)
004048a5          std::operator<<<std::char_traits<char> >(&std::cout, "\nEnter the magic word: ")
004048bb          std::operator>><char>(&std::cin, &var_48)
004048ce          if (validate_password(&var_48) != 0)
004048ce              break
0040493b          std::operator<<<std::char_traits<char> >(&std::cout, "Wrong password. Try Again :( \n\n")
004048e4      std::operator<<<std::char_traits<char> >(&std::cout, "Correct!")
004048e9      secret_function()
004048f8      std::istream::ignore(&std::cin)
00404911      std::operator<<<std::char_traits<char> >(&std::cout, "Press Enter to exit...")
00404920      std::istream::get(&std::cin)
00404951      std::string::~string(&var_48)
0040495c      *(fsbase + 0x28)
00404965      if (rax == *(fsbase + 0x28))
00404991          return 0
00404987      __stack_chk_fail()
00404987      noreturn
```

Fig. 34: Disassembled Code from Main Function

7.5 Examining the Secret Function

To analyze the function "secret_function", double click where it is called in main (at address 0x004048e9). Once double clicked, we should see something similar to Fig. 35 . By observing this function in Binary Ninja, we can see that when the secret function is called, the string "You have completed your first RE" (some characters are omitted in the figure).

```
00404795  int64_t* secret_function()

004047b8      return std::operator<<<std::char_traits<char> >(&std::cout, " You have completed your first R…")
```

Fig. 35: Function secret_function

```
004047b9  uint64_t validate_password(int64_t* arg1)

004047ca      void* fsbase
004047ca      int64_t rax = *(fsbase + 0x28)
004047fa      void var_48
004047fa      std::string::string<std::allocator<char> >(&var_48, "B3@rCatz")
00404819      int32_t rax_2 = _ZSteqIcEN9__gnu_cxx11__...gIS3_St11char_traitsIS3_ESaIS3_EEESE_(arg1, &var_48)
00404828      std::string::~string(&var_48)
00404833      *(fsbase + 0x28)
0040483c      if (rax == *(fsbase + 0x28))
00404868          return zx.q(rax_2)
0040485e      __stack_chk_fail()
0040485e      noreturn
```

Fig. 36: Function validate_password

7.6 Analyzing the Password Validation

Next, we will analyze the function validate_password. We return to the main function, select the function "validate_password" at address 0x004048ce, and then double click the function. We should see the dissembled code similar as in Fig. 35.

At this point, we can observe that the return type of this function is marked as “uint64_t” by Binary Ninja while the return type of this function is supposed/suspected to be bool (given the logic of this function). This suggests that Binary Ninja was unable to determine the correct return type on its own given this function. In such a case, we can manually adjust the return type.

Specifically, we can move to the “uint64_t” in the function signature in Fig. 36 at address 0x004047b9 and right click. This will open a drop-down window. We can search the “Change Type” in the drop down window and select it. This will open another window as in Fig. 37. Now, we can change “uint64_t” to “bool”, and select “Accept”. We can observe the code after this change in Fig. 38.

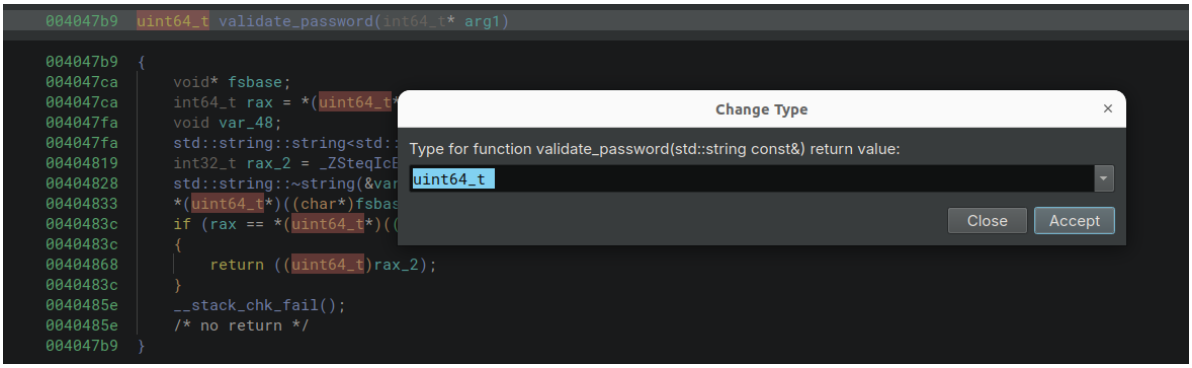


Fig. 37: Change return type

Fig. 38: Function validate_password (after changing the return type)

Even without the return type change, we can observe that a hard-coded string “B3@rCatz” was assigned to var_48, which is then compared with a variable arg1. This suggests that “B3@rCatz” is likely the hard-coded password in this example.

7.7 Patching the Code in Binary Ninja

In this section, we will try to modify the original binary and bypass the conditional jump instruction in the binary so that any input would be valid (i.e., printing "Correct! You have completed your first RE!"). Instead of using Ghidra, we will use Binary Ninja.

When we patch a binary, we edit the original instructions of the file and insert new instructions. This can be done by overwriting an instruction with a NOP instruction or inserting a whole new function! For now, we will consider the case of patching one instruction with one (or a few) NOP instructions. Therefore, when the patched binary is executed, the program will skip the old instructions and run the NOP instruction instead.

To carry out patching in Binary Ninja, we first must identify our target bytes. Our goal is to make the program accepting any password. As a result, we need to patch the bytes associated with the jump instruction executing the password check. Binary Ninja makes the identification of this instruction easy as we can observe the C code at the same time. In the code window, go back to the main function (using the forward and backward arrows, you should be able to traverse to the main function quickly! If not, search in the function list). Now, double click on line the corresponds to the conditional if statement, for our example, this is at address 0x004048ce, as shown in Figure 39. Doing so will highlight the assembly instructions that correspond to the conditional if statement in dissembled C code.

Fig. 39: je instruction associated with the if statement

Next, we modify the `je` instruction into a `nop` instruction. To do so, right click the `je` instruction and select "patch" in the window. Next, insert a new instruction `nop` as shown in Fig. 40.

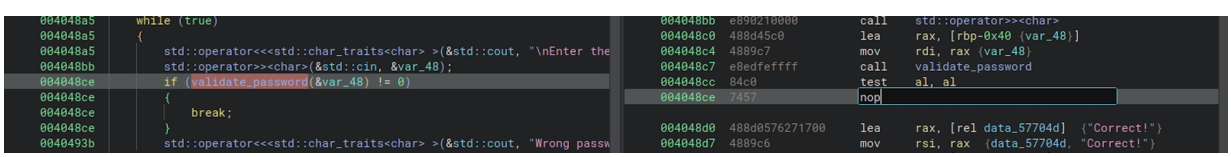


Fig. 40: Modify `je` instruction to `nop` instruction

Once we complete with this step, click anywhere outside of the instruction box, and we have a patched binary! The dissembled code in the main function will be updated to reflect this change, and we should observe similar as in Fig. 41.

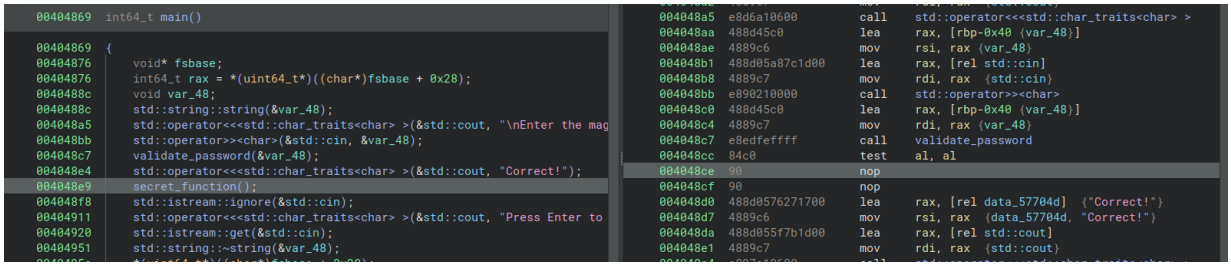


Fig. 41: The binary after the patching is completed

Now we can export the patched binary file and test it. To export the binary, we select "File→Save As", this will open a window. We name the patched binary as "EX_patched" in this example and click save.

To run it in the terminal, we first make it executable by running a command `chmod +x EX_patched`, then we run it with `./EX_patched`. We should obverse that any password will be accepted and show "Correct! You have completed your first RE!"

8 Conclusion

In conclusion, we are able to identify, analyze, and modify our first binary using Ghidra's reverse engineering capabilities. This example demonstrates the fundamental techniques for static analysis, identifying call functions, and binary patching. The workflow presented here provides a foundation for more difficult task in reverse engineering along the way.

There are many resources online. We only list a few at the end of this document [1–3] in case one would like to explore more resources related to (software) reverse engineering. Besides software reverse engineering, hardware reverse engineering is also a critical topic and even harder to learn. We do encourage students to explore hardware reverse engineering as well.

References

1. https://crackmes.one/.
2. https://github.com/nationalsecurityagency/ghidra.
3. https://picoctf.org/about.html.